

Büyük Ölçekli Akademik Atıf Ağlarının Analizi, Görselleştirilmesi ve K-Core Ayrışımı Üzerine Bir Uygulama

Burak Çakır

Bilgisayar Mühendisliği Bölümü

Kocaeli Üniversitesi

Kocaeli, Türkiye

220201003

Abstract—Bu çalışma, akademik literatürdeki makaleler arasındaki karmaşık atıf ilişkilerini modellemek, analiz etmek ve görselleştirmek amacıyla geliştirilen masaüstü tabanlı bir yazılım sistemini sunmaktadır. Proje, yapılandırılmamış veya yarı yapılandırılmış (JSON) verilerin işlenerek yönlü graf yapısına dönüştürülmesi ve bu yapı üzerinde graf teorisi metriklerinin uygulanması temeline dayanmaktadır. Geliştirilen uygulama, Java Swing kütüphanesi kullanılarak kullanıcı etkileşimli bir arayüz sunmaktadır. Sistem; H-Index, H-Core, H-Median gibi bibliyometrik hesaplamaların yanı sıra, ağ topolojisindeki kritik düğümleri belirlemek için "Betweenness Centrality" ve sıkı bağlı toplulukları tespit etmek için "K-Core Decomposition" algoritmalarını içermektedir. Görselleştirme aşamasında, düğümlerin çakışmasını önleyen ve okunabilirliği artıran "Force-Directed" (İtme Kuvveti) yerleşim algoritması uygulanmıştır. Deneysel sonuçlar, geliştirilen aracın 1000 düğümlü ağlarda bile yüksek performansla çalıştığını ve akademik etki analizinde araştırmacılara görsel bir içgörü sağladığını göstermektedir.

Index Terms—Graf Teorisi, Sosyal Ağ Analizi, H-Index, K-Core Ayrışımı, Betweenness Centrality, Veri Görselleştirme, Java.

I. GİRİŞ

Bilimsel üretimin hızla artmasıyla birlikte, akademik yayınların etkisi ve birbirleriyle olan ilişkilerinin analizi karmaşık bir hal almıştır. Geleneksel yöntemler (örneğin sadece atıf sayısına bakmak), bir yayının ağ içindeki gerçek etkisini veya hangi alt disiplinleri birbirine bağladığını göstermekte yetersiz kalabilmektedir. Bu bağlamda, Graf Teorisi ve Sosyal Ağ Analizi (SNA) yöntemleri, bibliyometrik verilerin anlamlandırılmasında güçlü araçlar sunmaktadır.

Bu proje kapsamında geliştirilen yazılım, ham atıf verilerini görsel bir ağ haritasına dönüştürmeyi hedeflemektedir. Projenin ana motivasyonu, sadece sayısal veriler sunmak yerine, araştırmacılara makaleler arasındaki ilişki ağını (citation network) interaktif bir şekilde keşfetme imkanı tanımaktır. Özellikle K-Core ayrışımı gibi ileri düzey tekniklerin entegrasyonu, literatürdeki ana akım çalışmaları (çekirdek yayınları) tespit etmeyi kolaylaştırmaktadır.

II. LİTERATÜR TARAMASI VE ARKA PLAN

Atıf ağlarının analizi, literatürde geniş yer bulan bir çalışmadır. Hirsch tarafından 2005 yılında önerilen H-Index [1],

bir araştırmacının hem üretkenliğini hem de atıf etkisini tek bir sayıyla ifade etmesi nedeniyle standart haline gelmiştir. Ancak H-Index, ağın yapısal özelliklerini (topolojisini) göz ardı eder.

Mark Newman ve diğerlerinin çalışmaları [2], ağlardaki topluluk yapılarının (community detection) önemini ortaya koymuştur. Bu bağlamda "K-Core Decomposition" yöntemi, ağın merkezindeki en yoğun bağlı kümeyi bulmak için etkili bir filtreleme yöntemi olarak kullanılmaktadır. Ayrıca Brandes [3] tarafından geliştirilen algoritmalar sayesinde, "Betweenness Centrality" gibi hesaplama maliyeti yüksek metriklerin büyük ağlarda uygulanabilirliği artmıştır. Bu proje, teorik literatürdeki bu metrikleri pratik ve görsel bir yazılım aracı üzerinde birleştirmeyi amaçlamaktadır.

III. SİSTEM MİMARISI VE TASARIM

Uygulama, Nesne Yönelimli Programlama (OOP) prensipleri çerçevesinde, modüler ve genişletilebilir bir yapıda tasarlanmıştır.

A. Sınıf Tasarımı (Class Design)

Sistemin omurgasını oluşturan temel sınıflar şunlardır:

- **Makale:** Veri modelini temsil eder. ID, Başlık, Yazar, Yıl gibi niteliklerin yanı sıra; references (giden kenarlar) ve citedBy (gelen kenarlar) listelerini tutarak graf yapısını oluşturur. Ayrıca analiz sonuçları için isKCore ve betweenness alanlarına sahiptir.
- **Grafİslemleri:** Ham verinin işlenmesi, ID ve Referans eşleştirmelerinin yapılarak grafın (düğümler ve kenarlar) bellekte oluşturulmasından sorumludur.
- **Hesaplama:** Projenin beynidir. H-Index, K-Core ve Centrality algoritmalarını statik metotlar halinde barındırır.
- **GrafPanel:** JPanel sınıfından türetilmiştir. Graphics2D kütüphanesini kullanarak düğümleri, kenarları ve metinleri ekrana çizer. Çakışma önleme algoritması burada çalışır.
- **Arayüz:** Kullanıcı etkileşimlerini (Butonlar, Textfield'lar) yöneten ana pencere sınıfıdır.

B. Veri Yapıları

Projede performans optimizasyonu için çeşitli veri yapıları kullanılmıştır:

- **HashMap:** Makale ID'lerinden nesnelere $O(1)$ karmaşıklığında erişmek için kullanılmıştır.
- **ArrayList:** Düğümlerin komşuluk listelerini (Adjacency List) tutmak için tercih edilmiştir.
- **Linked List Mantığı:** Proje isterleri gereği, makaleler ID sırasına göre yeşil kenarlarla bağlanarak sanal bir bağlı liste yapısı da graf üzerine entegre edilmiştir.

IV. ALGORİTMALAR VE YÖNTEM

A. H-Index ve H-Core Hesaplaması

H-Index hesabı için makaleye atıf yapan yayınlar, atıf sayılarına göre büyükten küçüğe sıralanır. Sıralı listede, atıf sayısı sıra numarasına eşit veya büyük olan son nokta H-Index olarak belirlenir. Bu noktaya kadar olan makaleler H-Core kümesini oluşturur.

B. K-Core Decomposition Algoritması

K-Core analizi, grafi "soğan kabuğu" gibi soyarak en yoğun çekirdeğe ulaşmayı hedefler. Algoritmanın çalışma mantığı şöyledir: 1. Tüm düğümlerin dereceleri (degree) hesaplanır. 2. Derecesi k 'dan küçük olan düğümler ve onlara bağlı kenarlar graftan silinir. 3. Silme işlemi komşuların derecesini düşürecek için, şartı sağlamayan yeni düğümler oluşabilir. Bu yüzden işlem, silinecek düğüm kalmayana kadar iteratif olarak tekrarlanır (Algoritma 1).

Algorithm 1 K-Core Decomposition (Java Mantığı)

Require: Graf Düğümleri (V), Eşik Değer (k)

Ensure: K-Core Kümesi İşaretlenmiş Graf

```
1:  $Degrees \leftarrow$  Her düğümün güncel derecesini hesapla
2:  $Changed \leftarrow \text{TRUE}$ 
3: while  $Changed$  is TRUE do
4:    $Changed \leftarrow \text{FALSE}$ 
5:    $ToRemove \leftarrow$  Empty List
6:   for all  $node$  in  $V$  do
7:     if  $Degrees[node] < k$  AND  $node$  is active then
8:       Add  $node$  to  $ToRemove$ 
9:     end if
10:  end for
11:  if  $ToRemove$  is not Empty then
12:     $Changed \leftarrow \text{TRUE}$ 
13:    for all  $node$  in  $ToRemove$  do
14:      Mark  $node$  as inactive
15:      for all  $neighbor$  of  $node$  do
16:         $Degrees[neighbor] \leftarrow Degrees[neighbor] - 1$ 
17:      end for
18:    end for
19:  end if
20: end while
21: return Aktif kalan düğümleri işaretle ( $isKCore = \text{true}$ )
```

C. Betweenness Centrality

Ağdaki bilgi akışını kontrol eden düğümleri bulmak için, yönsüz grafa dönüştürülen yapı üzerinde BFS (Breadth-First Search) tabanlı en kısa yol algoritması çalıştırılmıştır. Her düğümden başlatılan BFS ile o düğümden geçen en kısa yolların oranı (sigma) biriktirilerek skor hesaplanmıştır.

D. Force-Directed Yerleşim (Çakışma Önleme)

Görselleştirme sırasında düğümlerin üst üste binmesini engellemek için basit bir fizik motoru simüle edilmiştir. Her düğüm bir "yükü parçacık" gibi davranır ve birbirine çok yaklaşan düğümler ($d < 60px$) birbirini iter.

$$x_{yeni} = x_{eski} + (dx/d) \cdot F_{itme}$$

Bu işlem her çizim döngüsünde tekrarlanarak grafiğin kendi kendine organize olması sağlanır.

V. DENEYSEL SONUÇLAR VE UYGULAMA

Geliştirilen yazılım, 1000 makalelik test veri seti üzerinde çalıştırılmış ve aşağıda detaylandırılan senaryolar test edilmiştir.

A. Arayüz ve Genel İstatistikler

Uygulama açılışında veri seti taranarak genel istatistikler (Toplam makale: 1000, Toplam Referans: 1590) ve en popüler yayınlar başarıyla raporlanmıştır (Şekil 1).

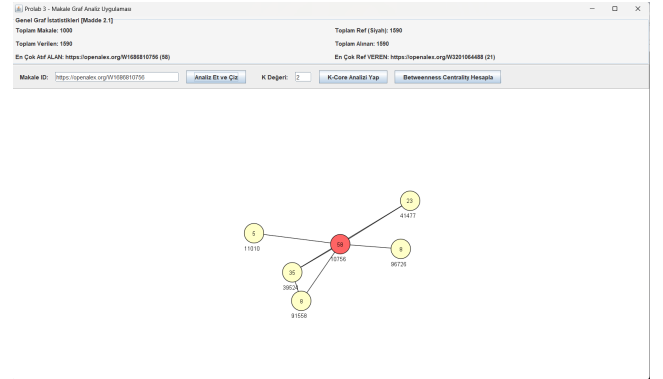


Fig. 1. Uygulama ana ekranı. Üst panelde genel ağ istatistikleri ve rekor sahibi makaleler listelenmektedir.

B. H-Index ve H-Core Görselleştirmesi

Kullanıcı belirli bir makale ID'si arattığında, sistem o makalenin H-Index skorunu (Örn: 58 atıf alan bir makale için) hesaplamış ve ilgili H-Core kümesini kırmızı merkez etrafında sarı düğümlerle görselleştirmiştir (Şekil 2).

C. K-Core Analizi Bulguları

Uygulamanın en güçlü özelliklerinden biri olan K-Core analizi, $k = 2$ değeri için test edilmiştir. Algoritma, ağdaki zayıf bağlantıları (derecesi 1 olanları) filtreleyerek, birbirine sıkı sıkıya bağlı çekirdek yapıyı ortaya çıkarmıştır. Bu yapılar arayüzde **mor renk** ve kalın mavi çizgilerle vurgulanmıştır (Şekil 3).

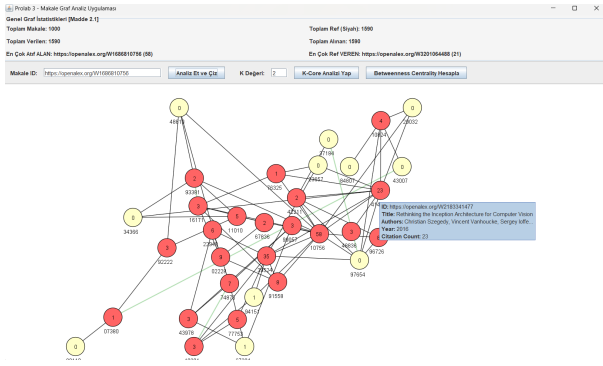


Fig. 2. Tekil makale analizi: Seçilen makale (Kırmızı) ve ona atfı yapan H-Core kümesi (Sarı).

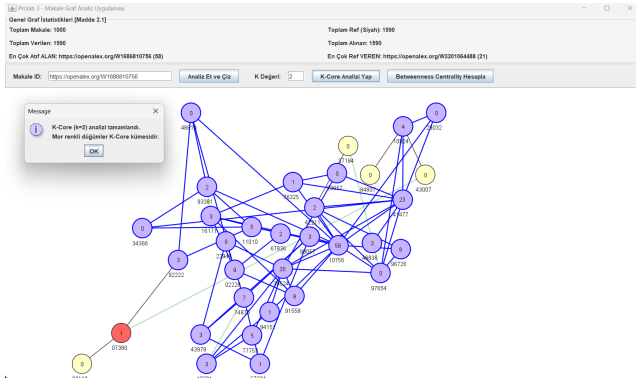


Fig. 3. K=2 için K-Core ayrışımı sonucu. Mor düğümler, ağıın en dayanıklı ve birbirine bağlı çekirdek kümesini temsil etmektedir.

D. Betweenness Centrality Listesi

Yapılan analizde, en yüksek Betweenness skoruna sahip makaleler tespit edilmiştir. Şekil 4'de görülen listedeki makaleler, ağıın farklı kümelerini birbirine bağlayan "köprü" niteliğindeki yayınlardır.

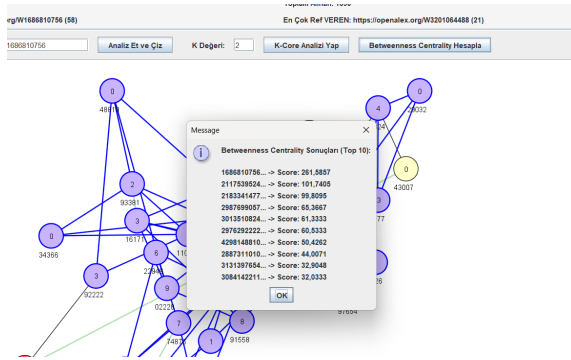


Fig. 4. Betweenness Centrality algoritması sonucu elde edilen en yüksek skorlu (en merkezi) makalelerin listesi.

VI. TARTIŞMA VE SONUÇ

Bu proje çalışması sonucunda, büyük ölçekli bibliyometrik verilerin analizi için kapsamlı bir masaüstü uygulaması

geliştirilmiştir. Elde edilen sonuçlar ışığında şu çıkarımlar yapılabilir:

- 1) **Görselleştirmenin Önemi:** Sayısal listeler yerine graf tabanlı görselleştirme, araştırmacıların atf desenlerini (citation patterns) daha hızlı kavramasını sağlamaktadır. Özellikle "Force-Directed" yerleşim, düğüm yoğunluğunu yönetmede başarılı olmuştur.
- 2) **K-Core Etkisi:** K-Core ayrışımı, literatürdeki "gürültüyü" (az atf alan veya izole yayınları) temizleyerek ana akım tartışmaları ortaya çıkarmada etkili bir yöntemdir.
- 3) **Performans:** Java'nın koleksiyon yapılarının (Map, List) etkin kullanımı sayesinde, algoritmaların çalışma süresi kabul edilebilir sınırlar içinde kalmıştır.

Gelecek çalışmalarda, grafiğin 3 boyutlu (3D) uzayda görselleştirilmesi ve veritabanı (SQL/NoSQL) entegrasyonu ile daha büyük veri setlerinin (örn: 1 milyon makale) desteklenmesi hedeflenmektedir.

REFERENCES

- [1] J. E. Hirsch, "An index to quantify an individual's scientific research output," *Proc. Nat. Acad. Sci.*, vol. 102, no. 46, pp. 16569–16572, 2005.
- [2] S. B. Seidman, "Network structure and minimum degree," *Social Networks*, vol. 5, no. 3, pp. 269–287, 1983.
- [3] U. Brandes, "A faster algorithm for betweenness centrality," *Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001.
- [4] M. E. J. Newman, "Networks: An Introduction," Oxford University Press, 2010.
- [5] Oracle, "Java Swing Documentation Graphics2D API," [Online]. Erişim: <https://docs.oracle.com/javase/tutorial/uiswing/>